

# Tugas 12: Praktikum & Praktikum Mandiri 12

Pandu Linggar Kumara - 0110221277,  
Link GitHub - [https://github.com/PanduLgg/M\\_Learning.git](https://github.com/PanduLgg/M_Learning.git)

<sup>1</sup> Teknik Informatika, STT Terpadu Nurul Fikri, Depok

\*E-mail: [pandulinggar1@gmail.com](mailto:pandulinggar1@gmail.com)

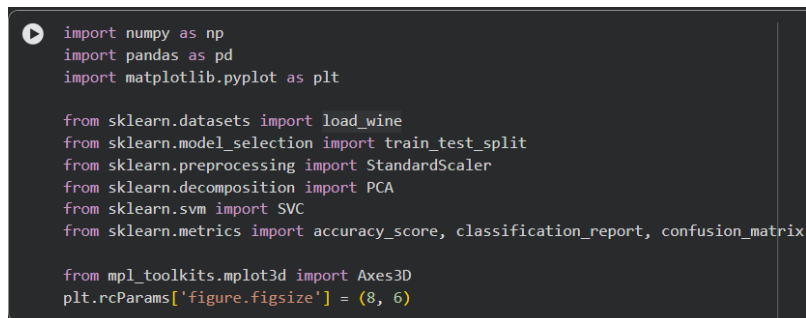
## Abstract.

Dataset medis umumnya memiliki banyak fitur sehingga dapat meningkatkan kompleksitas model klasifikasi. Pada praktikum ini, Principal Component Analysis (PCA) digunakan untuk mereduksi dimensi pada dataset Breast Cancer yang terdiri dari 569 data dengan 30 fitur numerik. Proses yang dilakukan meliputi standarisasi data, pembuatan model Support Vector Machine (SVM) tanpa PCA sebagai pembandingan, serta penerapan PCA untuk mengurangi jumlah fitur. Model SVM kemudian dilatih kembali menggunakan data hasil PCA dan dievaluasi performanya. Hasil praktikum menunjukkan bahwa PCA mampu mereduksi jumlah fitur secara signifikan tanpa menurunkan akurasi model secara berarti. Model dengan PCA memiliki performa yang relatif sebanding dengan model tanpa PCA, namun lebih sederhana dan efisien. Oleh karena itu, PCA dapat digunakan sebagai teknik reduksi dimensi yang efektif pada dataset medis berdimensi tinggi.

## 1. PRAKTIKUM 12

### 1.1 Import Library dan Model

Sel ini berfungsi untuk Mengimport Library dan Model yang dibutuhkan, mengimpor library untuk mengolah data berbentuk tabel.



```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

from mpl_toolkits.mplot3d import Axes3D
plt.rcParams['figure.figsize'] = (8, 6)
```

**Gambar 1.1.** Proses ini hanya perlu dilakukan satu kali per sesi.

## 1.2 Memanggil Data set dari Gdrive dan Membaca file .CSV menggunakan Pandas

Sel ini menggunakan library Pandas untuk membaca file data, yang diinginkan dan membaca file wine dari direktori kaggle dan menyimpannya ke DataFrame, dan melihat informasi data 5 baris atas.

```
(3) wine = load_wine()

df = pd.DataFrame(wine.data, columns=wine.feature_names)
df['target'] = wine.target

df.head()
```

|   | alcohol | malic_acid | ash  | alcalinity_of_ash | magnesium | total_phenols | flavanoids | nonflavanoid_phenols | proanthocyanins | color_intensity | hue  | od280/od315_of_diluted_wines | proline |
|---|---------|------------|------|-------------------|-----------|---------------|------------|----------------------|-----------------|-----------------|------|------------------------------|---------|
| 0 | 14.23   | 1.71       | 2.43 | 15.6              | 127.0     | 2.80          | 3.06       | 0.28                 | 2.29            | 5.64            | 1.04 | 3.92                         | 1065    |
| 1 | 13.20   | 1.78       | 2.14 | 11.2              | 100.0     | 2.65          | 2.76       | 0.26                 | 1.28            | 4.38            | 1.05 | 3.40                         | 1050    |
| 2 | 13.16   | 2.36       | 2.67 | 18.6              | 101.0     | 2.80          | 3.24       | 0.30                 | 2.81            | 5.68            | 1.03 | 3.17                         | 1185    |
| 3 | 14.37   | 1.95       | 2.50 | 16.8              | 113.0     | 3.85          | 3.49       | 0.24                 | 2.18            | 7.80            | 0.86 | 3.45                         | 1480    |
| 4 | 13.24   | 2.59       | 2.87 | 21.0              | 118.0     | 2.80          | 2.69       | 0.39                 | 1.82            | 4.32            | 1.04 | 2.93                         | 735     |

Gambar 1.2. Proses ini hanya perlu dilakukan satu kali per sesi.

## 1.3 Mencari informasi data yang ada pada file

Analisis eksplorasi awal (Exploratory Data Analysis / EDA) mencakup pemeriksaan tipe data, jumlah data, nilai kosong, data duplikat, serta distribusi label (target class).

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 178 entries, 0 to 177
Data columns (total 14 columns):
 #   Column                                Non-Null Count  Dtype
---  ---                                ---
 0   alcohol                             178 non-null    float64
 1   malic_acid                         178 non-null    float64
 2   ash                                178 non-null    float64
 3   alcalinity_of_ash                  178 non-null    float64
 4   magnesium                          178 non-null    float64
 5   total_phenols                      178 non-null    float64
 6   flavanoids                         178 non-null    float64
 7   nonflavanoid_phenols               178 non-null    float64
 8   proanthocyanins                    178 non-null    float64
 9   color_intensity                    178 non-null    float64
10   hue                                178 non-null    float64
11   od280/od315_of_diluted_wines       178 non-null    float64
12   proline                            178 non-null    float64
13   target                             178 non-null    int64
dtypes: float64(13), int64(1)
memory usage: 19.6 KB
```

Gambar 1.3. df.info

## 1.4 Informasi Dataset

menampilkan rangkuman statistik deskriptif dari setiap kolom numerik.

```
df.describe()
```

|       | alcohol    | malic_acid | ash        | alcalinity_of_ash | magnesium  | total_phenols | flavanoids | nonflavanoid_phenols | proanthocyanins | color_intensity | hue        |
|-------|------------|------------|------------|-------------------|------------|---------------|------------|----------------------|-----------------|-----------------|------------|
| count | 178.000000 | 178.000000 | 178.000000 | 178.000000        | 178.000000 | 178.000000    | 178.000000 | 178.000000           | 178.000000      | 178.000000      | 178.000000 |
| mean  | 13.000618  | 2.336348   | 2.366517   | 19.494944         | 99.741573  | 2.295112      | 2.029270   | 0.361854             | 1.590899        | 5.058090        | 0.957449   |
| std   | 0.811827   | 1.117146   | 0.274344   | 3.339564          | 14.282484  | 0.625851      | 0.998859   | 0.124453             | 0.572359        | 2.318286        | 0.228572   |
| min   | 11.030000  | 0.740000   | 1.360000   | 10.600000         | 70.000000  | 0.980000      | 0.340000   | 0.130000             | 0.410000        | 1.280000        | 0.480000   |
| 25%   | 12.362500  | 1.602500   | 2.210000   | 17.200000         | 88.000000  | 1.742500      | 1.205000   | 0.270000             | 1.250000        | 3.220000        | 0.782500   |
| 50%   | 13.050000  | 1.865000   | 2.360000   | 19.500000         | 98.000000  | 2.355000      | 2.135000   | 0.340000             | 1.555000        | 4.690000        | 0.965000   |
| 75%   | 13.677500  | 3.082500   | 2.557500   | 21.500000         | 107.000000 | 2.800000      | 2.875000   | 0.437500             | 1.950000        | 6.200000        | 1.120000   |
| max   | 14.830000  | 5.800000   | 3.230000   | 30.000000         | 162.000000 | 3.880000      | 5.080000   | 0.660000             | 3.580000        | 13.000000       | 1.710000   |

Gambar 1.4. df.describe

### 1.5 Cek Missing Value

Mengecek jumlah data kosong untuk setiap kolom.

```
df.isnull().sum()
```

|                              | 0 |
|------------------------------|---|
| alcohol                      | 0 |
| malic_acid                   | 0 |
| ash                          | 0 |
| alcalinity_of_ash            | 0 |
| magnesium                    | 0 |
| total_phenols                | 0 |
| flavanoids                   | 0 |
| nonflavanoid_phenols         | 0 |
| proanthocyanins              | 0 |
| color_intensity              | 0 |
| hue                          | 0 |
| od280/od315_of_diluted_wines | 0 |
| proline                      | 0 |
| target                       | 0 |

**Gambar 1.5.** Tidak ada missing value

### 1.6 Cek Label Target

Distribusi label target cukup seimbang, memungkinkan penerapan algoritma klasifikasi seperti SVM tanpa perlu melakukan penyeimbangan data tambahan..

```
print("Nama Kelas:", wine.target_names)
df['target'].value_counts()
```

Nama Kelas: ['class\_0' 'class\_1' 'class\_2']

|        | count |
|--------|-------|
| target |       |
| 1      | 71    |
| 0      | 59    |
| 2      | 48    |

dtype: int64

**Gambar 1.6.** Cek Label Target

### 1.7 Label dan Fitur

Tahap ini bertujuan untuk memisahkan data fitur (X) dan label target (y) dari dataset.

```
X = wine.data
y = wine.target

print("Shape of X:", X.shape) #178, 13
print("Shape of y:", y.shape) # 178

Shape of X: (178, 13)
Shape of y: (178,)
```

**Gambar 1.7.** Pemisahan Fitur dan lebel

### 1.8 Pembagian Data Latih dan Data Uji

Setelah data fitur (X) dan label (y) dipisahkan, tahap berikutnya adalah membagi dataset menjadi data latih (training data) dan data uji (testing data).

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Shape X_train:", X_train.shape)
print("Shape X_test :", X_test.shape)
```

... Shape X\_train: (142, 13)  
Shape X\_test : (36, 13)

Gambar 1.8. Data Latih dan Data Uji

### 1.9 Standardisasi Data

Untuk memastikan bahwa seluruh fitur berada pada skala yang sebanding, sehingga tidak ada fitur yang mendominasi hasil komponen utama hanya karena memiliki nilai numerik yang lebih besar

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

X_train_scaled[:5]
```

```
array([[ 0.38580089, -0.63787118,  1.77666817, -1.22453161,  0.69643032,
         0.52686525,  0.73229212, -0.1695489 , -0.41578344, -0.16746725,
         0.62437819,  0.2529082 ,  0.46772474],
       [ 0.94851892, -0.76544542,  1.25317383,  0.85328406,  0.09178497,
         1.17279546,  1.33318146, -0.59045701,  1.34974202,  0.30530313,
         1.06715537,  0.15104809,  1.81576773],
       [ 0.52335419, -0.51940939,  0.9540342 , -1.04643312, -0.44567755,
         0.93057163,  1.006382 , -0.1695489 , -0.26000178, -0.081509 ,
        -0.12834302,  0.89317174,  1.51620262],
       [ 0.97352861, -0.55585917,  0.16879269, -1.0761162 , -0.71440882,
         0.52686525,  0.81662747, -0.59045701,  0.36312485,  0.262324 ,
         0.8900445 ,  0.42752553,  1.93226527],
       [ 0.43582027,  0.82012009,  0.05661533,  0.55645325, -0.51286037,
        -0.55506784, -1.29175618,  0.75644894, -0.60618325,  1.47433535,
        -1.76661859, -1.43505932, -0.29783054]])
```

Gambar 1.9. scaler.fit\_transform

### 1.10 Model SVM tanpa PCA

Sebelum melakukan reduksi dimensi dengan PCA, terlebih dahulu dibuat model baseline menggunakan seluruh fitur asli (13 fitur)

```
svm_no_pca = SVC(kernel='rbf', gamma="scale", random_state=42)
svm_no_pca.fit(X_train_scaled, y_train)

y_pred_no_pca = svm_no_pca.predict(X_test_scaled)
acc_no_pca = accuracy_score(y_test, y_pred_no_pca)

print("Akurasi tanpa PCA:", acc_no_pca)
print("\nClassification Report (tanpa PCA):")
print(classification_report(y_test, y_pred_no_pca, target_names=wine.target_names))
#
```

Akurasi tanpa PCA: 0.9722222222222222

Classification Report (tanpa PCA):

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| class_0      | 1.00      | 1.00   | 1.00     | 12      |
| class_1      | 0.93      | 1.00   | 0.97     | 14      |
| class_2      | 1.00      | 0.90   | 0.95     | 10      |
| accuracy     |           |        | 0.97     | 36      |
| macro avg    | 0.98      | 0.97   | 0.97     | 36      |
| weighted avg | 0.97      | 0.97   | 0.97     | 36      |

Gambar 1.10. SVM tanpa PCA (Base line)

### 1.11 Penerapan PCA

Setelah data distandarisasi, tahap selanjutnya adalah menerapkan PCA (Principal Component Analysis) untuk mereduksi jumlah fitur dari 13 menjadi 3 komponen utama.

```
pca = PCA(n_components=3)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

print("Shape X_train_pca:", X_train_pca.shape)
print("Shape X_test_pca :", X_test_pca.shape)

Shape X_train_pca: (142, 3)
Shape X_test_pca : (36, 3)
```

**Gambar 1.11.** Reduksi dimensi

### 1.12 Menampilkan Variansi oleh Komponen PCA

Setelah PCA dilakukan, kita perlu mengetahui berapa persen informasi (variansi) dari dataset asli yang masih bisa dijelaskan oleh setiap komponen utama (Principal Component).

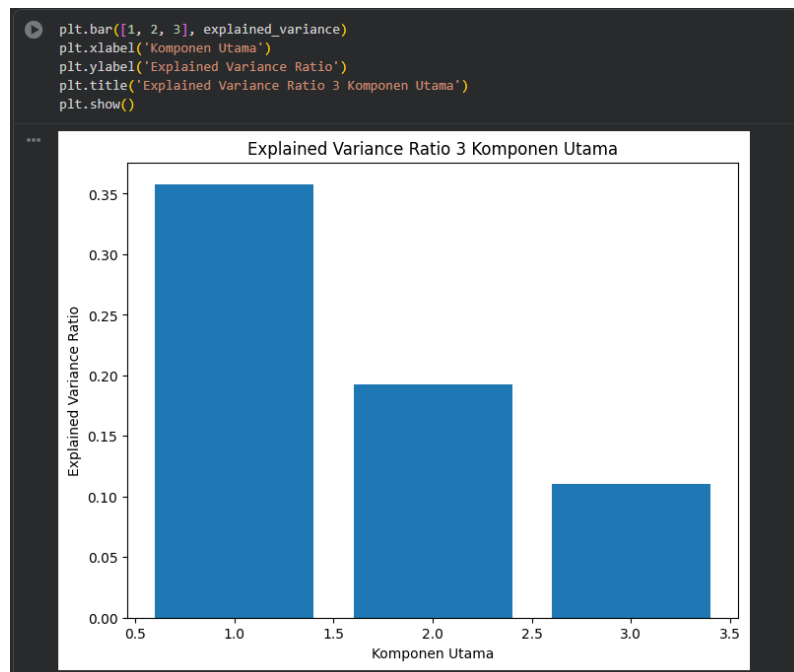
```
[16] explained_variance = pca.explained_variance_ratio_
      print("Explained Variance Ratio tiap komponen:", explained_variance)
      print("Total Explained Variance:", sum(explained_variance))

Explained Variance Ratio tiap komponen: [0.35792104 0.19270671 0.11019835]
Total Explained Variance: 0.6608261082211259
```

**Gambar 1.12.** Explained Variance Ratio

### 1.13 Visualisasi Explained Variance Ratio

Visualisasi Explained Variance Ratio dalam bentuk diagram batang (bar chart).



**Gambar 1.13.** Visualisasi

### 1.14 Membangun Model SVM dengan PCA

Melatih model SVM menggunakan hasil transformasi PCA, agar mengetahui apakah penggunaan PCA memengaruhi kinerja model klasifikasi.

```
svm_pca = SVC(kernel='rbf', gamma='scale', random_state=42)
svm_pca.fit(X_train_pca, y_train)

y_pred_pca = svm_pca.predict(X_test_pca)

acc_pca = accuracy_score(y_test, y_pred_pca)
print("Akurasi dengan PCA:", acc_pca)
print("\nClassification Report (dengan PCA):")
print(classification_report(y_test, y_pred_pca, target_names=wine.target_names))
#
```

Akurasi dengan PCA: 0.9722222222222222

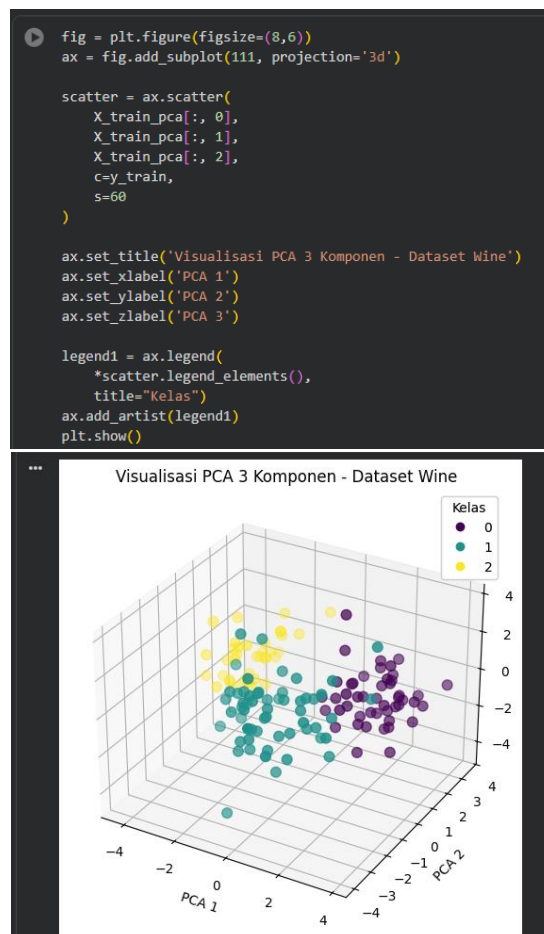
Classification Report (dengan PCA):

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| class_0      | 1.00      | 1.00   | 1.00     | 12      |
| class_1      | 0.93      | 1.00   | 0.97     | 14      |
| class_2      | 1.00      | 0.90   | 0.95     | 10      |
| accuracy     |           |        | 0.97     | 36      |
| macro avg    | 0.98      | 0.97   | 0.97     | 36      |
| weighted avg | 0.97      | 0.97   | 0.97     | 36      |

Gambar 1.14. Latih PCA

### 1.15 Visualisasi 3D PCA

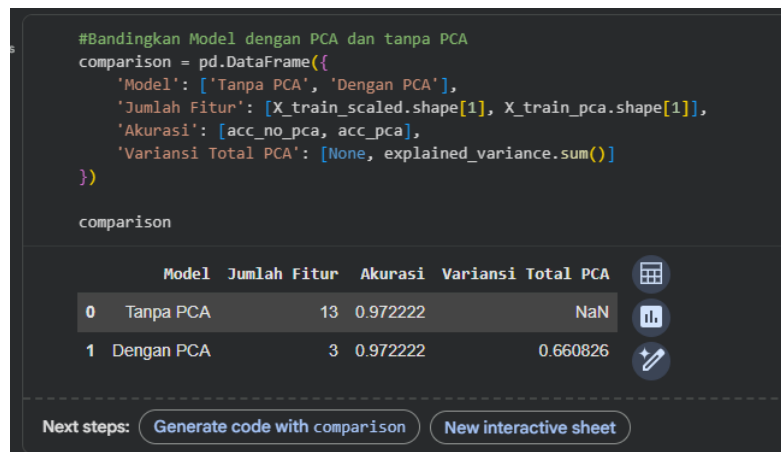
Setelah reduksi dimensi menggunakan PCA dengan 3 komponen lalu visualisasikan data dalam ruang tiga dimensi (3D).



Gambar 1.15. Visualisasi 3D PCA

### 1.16 Perbandingan Model SVM Tanpa PCA dan Dengan PCA

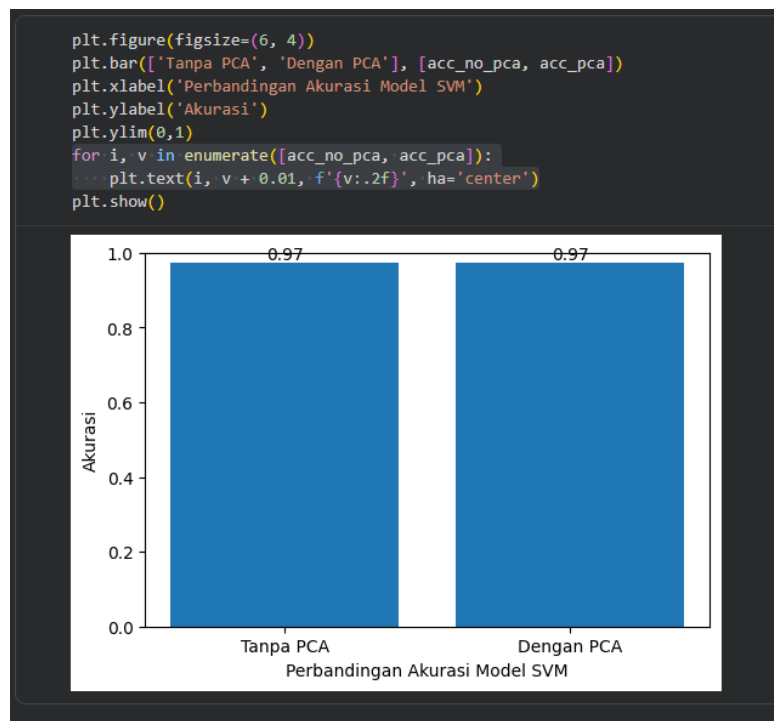
Jumlah fitur berkurang drastis dari 13 menjadi 3 komponen utama. Akurasi model tetap sama tinggi (97.22%), meskipun jumlah fitur direduksi hingga lebih dari 75%, Hal ini membuktikan bahwa PCA dapat menyederhanakan data tanpa kehilangan performa model.



Gambar 1.16. PCA vs No PCA

### 1.17 Visualisasi Perbandingan

Kedua model (tanpa PCA dan dengan PCA) memiliki akurasi yang identik (0.97), Model dengan PCA justru lebih efisien, karena menggunakan lebih sedikit fitur dan lebih cepat dalam pelatihan.



Gambar 1.17. Visualisasi Perbandingan

## 2. PRAKTIKUM MANDIRI (MANDIRI)

### 2.1 Import Library

Tahap ini menyiapkan seluruh pustaka (library) yang dibutuhkan untuk pengolahan data, penerapan PCA, pembuatan model klasifikasi, dan visualisasi.

```
[1] 10s
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

plt.rcParams['figure.figsize'] = (8,6)
```

Gambar 2.1. Install Library

### 2.2 Load Dataset Breast Cancer

Dataset Breast Cancer dimuat dari pustaka sklearn, kemudian disusun dalam bentuk DataFrame agar mudah dianalisis. Dataset berhasil dimuat dengan: 569 data, 30 fitur, 2 kelas target (ganas dan jinak).

```
[2] cancer = load_breast_cancer()
X = cancer.data
y = cancer.target

feature_names = cancer.feature_names

df = pd.DataFrame(X, columns=feature_names)
df['target'] = y

df.head()
```

|   | mean radius | mean texture | mean perimeter | mean area | mean smoothness | mean compactness | mean concavity | mean concave points | mean symmetry | mean fractal dimension | ... | worst texture | worst perimeter | worst area | worst smoothness | worst compactness | worst concavity | worst concave points | worst symmetry | worst fractal dimension |
|---|-------------|--------------|----------------|-----------|-----------------|------------------|----------------|---------------------|---------------|------------------------|-----|---------------|-----------------|------------|------------------|-------------------|-----------------|----------------------|----------------|-------------------------|
| 0 | 17.99       | 10.38        | 122.80         | 1001.0    | 0.11840         | 0.27760          | 0.3001         | 0.14710             | 0.2419        | 0.07871                | ... | 17.33         | 184.60          | 2019.0     | 0.1622           | 0.6656            | 0.7119          | 0.2654               | 0.4601         | 0.11890                 |
| 1 | 20.57       | 17.77        | 132.90         | 1326.0    | 0.08474         | 0.07864          | 0.0869         | 0.07017             | 0.1812        | 0.05667                | ... | 23.41         | 158.80          | 1956.0     | 0.1238           | 0.1866            | 0.2416          | 0.1060               | 0.2750         | 0.08902                 |
| 2 | 19.69       | 21.25        | 130.00         | 1203.0    | 0.10960         | 0.15990          | 0.1974         | 0.12790             | 0.2069        | 0.05999                | ... | 25.53         | 152.50          | 1709.0     | 0.1444           | 0.4245            | 0.4504          | 0.2430               | 0.3613         | 0.08758                 |
| 3 | 11.42       | 20.38        | 77.58          | 386.1     | 0.14250         | 0.28390          | 0.2414         | 0.10520             | 0.2597        | 0.09744                | ... | 26.50         | 98.87           | 567.7      | 0.2098           | 0.8663            | 0.6869          | 0.2575               | 0.6638         | 0.17300                 |
| 4 | 20.29       | 14.34        | 135.10         | 1297.0    | 0.10030         | 0.13280          | 0.1980         | 0.10430             | 0.1809        | 0.05883                | ... | 16.67         | 152.20          | 1575.0     | 0.1374           | 0.2050            | 0.4000          | 0.1625               | 0.2364         | 0.07578                 |

5 rows x 31 columns

Gambar 2.2. Load Dataset



## 2.3 Eksplorasi Awal Dataset

Untuk memahami kondisi dan kualitas data sebelum diproses lebih lanjut.

```
df.info()

*** <class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 31 columns):
 #   Column                                  Non-Null Count  Dtype  
---  -
 0   mean radius                            569 non-null    float64
 1   mean texture                           569 non-null    float64
 2   mean perimeter                         569 non-null    float64
 3   mean area                             569 non-null    float64
 4   mean smoothness                        569 non-null    float64
 5   mean compactness                       569 non-null    float64
 6   mean concavity                         569 non-null    float64
 7   mean concave points                    569 non-null    float64
 8   mean symmetry                          569 non-null    float64
 9   mean fractal dimension                 569 non-null    float64
10  radius error                           569 non-null    float64
11  texture error                           569 non-null    float64
12  perimeter error                        569 non-null    float64
13  area error                             569 non-null    float64
14  smoothness error                       569 non-null    float64
15  compactness error                      569 non-null    float64
16  concavity error                        569 non-null    float64
17  concave points error                   569 non-null    float64
18  symmetry error                         569 non-null    float64
19  fractal dimension error                569 non-null    float64
20  worst radius                           569 non-null    float64
21  worst texture                           569 non-null    float64
22  worst perimeter                        569 non-null    float64
23  worst area                             569 non-null    float64
24  worst smoothness                       569 non-null    float64
25  worst compactness                      569 non-null    float64
26  worst concavity                        569 non-null    float64
27  worst concave points                   569 non-null    float64
28  worst symmetry                         569 non-null    float64
29  worst fractal dimension                 569 non-null    float64
30  target                                569 non-null    int64  
dtypes: float64(30), int64(1)
memory usage: 137.9 KB
```

Gambar 2.3. Info Dataset

## 2.4 Informasi Isi Dataset

Menampilkan informasi kolom, tipe data, dan jumlah data

```
df.describe()
```

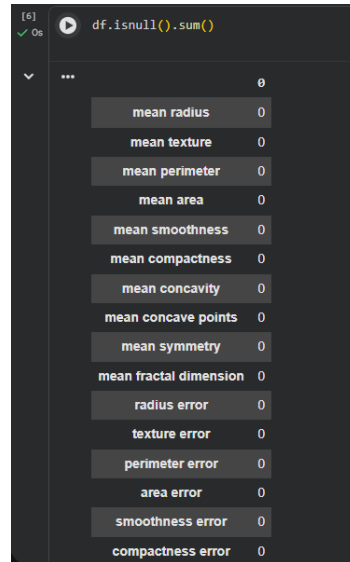
|       | mean radius | mean texture | mean perimeter | mean area   | mean smoothness | mean compactness | mean concavity | mean concave points | mean symmetry | mean fractal dimension | ... | worst texture | worst perimeter | worst area  |
|-------|-------------|--------------|----------------|-------------|-----------------|------------------|----------------|---------------------|---------------|------------------------|-----|---------------|-----------------|-------------|
| count | 569.000000  | 569.000000   | 569.000000     | 569.000000  | 569.000000      | 569.000000       | 569.000000     | 569.000000          | 569.000000    | 569.000000             | ... | 569.000000    | 569.000000      | 569.000000  |
| mean  | 14.127292   | 19.289649    | 91.969033      | 654.889104  | 0.096360        | 0.104341         | 0.088799       | 0.048919            | 0.181162      | 0.062798               | ... | 25.677223     | 107.261213      | 880.583128  |
| std   | 3.524049    | 4.301036     | 24.298981      | 351.914129  | 0.014064        | 0.052813         | 0.079720       | 0.038803            | 0.027414      | 0.007060               | ... | 6.146258      | 33.602542       | 569.356993  |
| min   | 6.981000    | 9.710000     | 43.790000      | 143.500000  | 0.052630        | 0.019380         | 0.000000       | 0.000000            | 0.106000      | 0.049960               | ... | 12.020000     | 50.410000       | 185.200000  |
| 25%   | 11.700000   | 16.170000    | 75.170000      | 420.300000  | 0.086370        | 0.064920         | 0.029560       | 0.020310            | 0.161900      | 0.057700               | ... | 21.080000     | 84.110000       | 515.300000  |
| 50%   | 13.370000   | 18.840000    | 86.240000      | 551.100000  | 0.095870        | 0.092630         | 0.061540       | 0.033500            | 0.179200      | 0.061540               | ... | 25.410000     | 97.660000       | 686.500000  |
| 75%   | 15.780000   | 21.800000    | 104.100000     | 782.700000  | 0.105300        | 0.130400         | 0.130700       | 0.074000            | 0.195700      | 0.066120               | ... | 29.720000     | 125.400000      | 1084.000000 |
| max   | 28.110000   | 39.280000    | 188.500000     | 2501.000000 | 0.163400        | 0.345400         | 0.426800       | 0.201200            | 0.304000      | 0.097440               | ... | 49.540000     | 251.200000      | 4254.000000 |

8 rows x 15 columns

Gambar 2.4. Info Dataset df.describe

## 2.5 Data Bersih

Dataset bersih, tidak memiliki missing value, dan distribusi kelas cukup seimbang sehingga siap digunakan untuk pelatihan model.



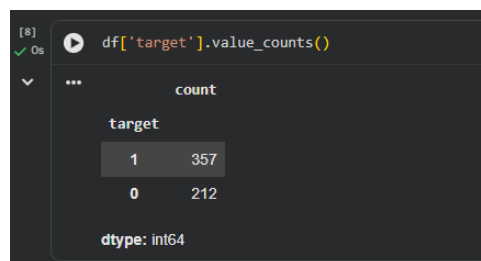
```
[6]: df.isnull().sum()
```

|                        | 0 |
|------------------------|---|
| mean radius            | 0 |
| mean texture           | 0 |
| mean perimeter         | 0 |
| mean area              | 0 |
| mean smoothness        | 0 |
| mean compactness       | 0 |
| mean concavity         | 0 |
| mean concave points    | 0 |
| mean symmetry          | 0 |
| mean fractal dimension | 0 |
| radius error           | 0 |
| texture error          | 0 |
| perimeter error        | 0 |
| area error             | 0 |
| smoothness error       | 0 |
| compactness error      | 0 |

**Gambar 2.5.** Cek data Null, dan duplicate

## 2.6 Cek Label Target

Distribusi label target cukup seimbang, memungkinkan penerapan algoritma klasifikasi seperti SVM tanpa perlu melakukan penyeimbangan data tambahan.



```
[8]: df['target'].value_counts()
```

| target | count |
|--------|-------|
| 1      | 357   |
| 0      | 212   |

dtype: int64

**Gambar 2.6.** Cek Target

## 2.7 Data dibagi menjadi data latih dan data uji

Model dapat diuji secara objektif pada data yang belum pernah dilihat sebelumnya.

```
[10] ✓ Os X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

**Gambar 2.7.** Split Data Uji dan Latih

## 2.8 Standardisasi Data

Nilai setiap fitur disesuaikan agar memiliki skala yang sama.

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

**Gambar 2.8.** Visualisasi Heat Map Folium

## 2.9 Model Baseline: SVM Tanpa PCA

Model SVM dibangun menggunakan seluruh fitur asli tanpa reduksi dimensi. Diperoleh nilai akurasi awal SVM dengan 30 fitur, yang digunakan sebagai acuan evaluasi.

```
[12] ✓ Os svm_no_pca = SVC(kernel='rbf', gamma='scale')
svm_no_pca.fit(X_train_scaled, y_train)

y_pred_no_pca = svm_no_pca.predict(X_test_scaled)

acc_no_pca = accuracy_score(y_test, y_pred_no_pca)
print("Akurasi SVM tanpa PCA:", acc_no_pca)
print(classification_report(y_test, y_pred_no_pca))
```

|                                           |           |        |          |         |  |
|-------------------------------------------|-----------|--------|----------|---------|--|
| Akurasi SVM tanpa PCA: 0.9824561403508771 |           |        |          |         |  |
|                                           | precision | recall | f1-score | support |  |
| 0                                         | 0.98      | 0.98   | 0.98     | 42      |  |
| 1                                         | 0.99      | 0.99   | 0.99     | 72      |  |
| accuracy                                  |           |        | 0.98     | 114     |  |
| macro avg                                 | 0.98      | 0.98   | 0.98     | 114     |  |
| weighted avg                              | 0.98      | 0.98   | 0.98     | 114     |  |

**Gambar 2.9.** Model Base Line No PCA

### 2.10 Penerapan PCA (Explained Variance)

Beberapa komponen awal sudah mampu menjelaskan sebagian besar variansi data, sehingga PCA layak digunakan untuk reduksi dimensi.

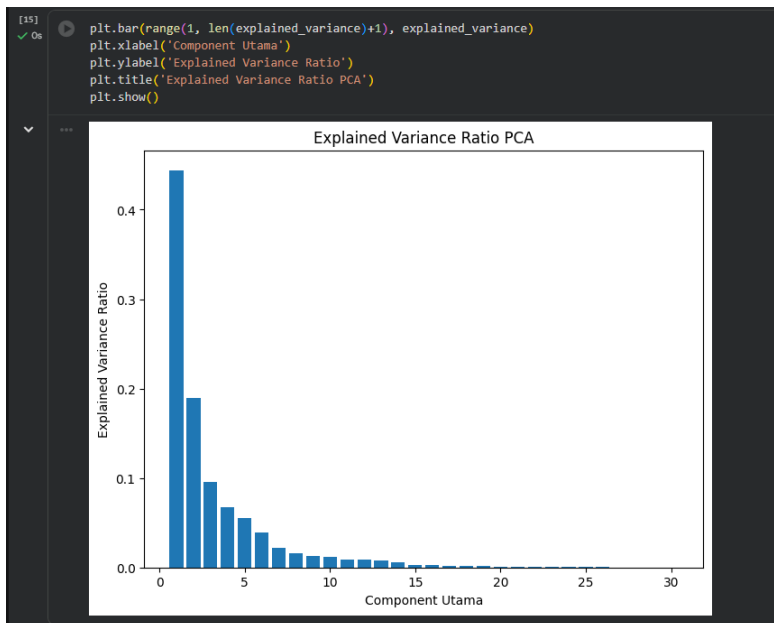
```
pca_full = PCA()
pca_full.fit(X_train_scaled)

explained_variance = pca_full.explained_variance_ratio_
```

**Gambar 2.10.** Menentukan jumlah komponen PCA yang optimal

### 2.11 Visualisasi 30 Fitur

Terdapat 30 Fitur Base Line



**Gambar 2.11.** Visualisasi Vitur Baseline

### 2.12 Reduksi Fitur

Jumlah fitur direduksi dari 30 menjadi 2 komponen utama. Data berhasil direpresentasikan dalam dua komponen utama yang masih mempertahankan informasi penting.

```
pca_2 = PCA(n_components=2)

X_train_pca = pca_2.fit_transform(X_train_scaled)
X_test_pca = pca_2.transform(X_test_scaled)

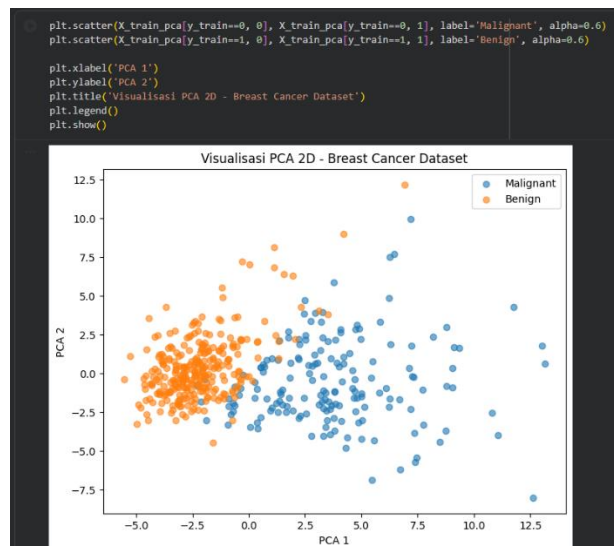
print("Total Variance (2 PC):", np.sum(pca_2.explained_variance_ratio_))

Total Variance (2 PC): 0.6335811006772492
```

**Gambar 2.12.** Model dengan PCA 2 Komponen

### 2.13 Visualisasi PCA 2D

Terlihat pemisahan kelas yang cukup jelas meskipun hanya menggunakan dua komponen utama.



**Gambar 2.13.** Visualisasi 2 Komponen PCA

### 2.14 Model SVM dengan PCA

Akurasi model dengan PCA relatif sebanding dengan model tanpa PCA, meskipun jumlah fitur jauh lebih sedikit.

```
svm_pca = SVC(kernel='rbf', gamma='scale')
svm_pca.fit(X_train_pca, y_train)

y_pred_pca = svm_pca.predict(X_test_pca)

acc_pca = accuracy_score(y_test, y_pred_pca)
print("Akurasi SVM dengan PCA:", acc_pca)
print(classification_report(y_test, y_pred_pca))
```

```
*** Akurasi SVM dengan PCA: 0.9473684210526315
      precision    recall  f1-score   support

     0       0.95       0.90       0.93         42
     1       0.95       0.97       0.96         72

 accuracy
macro avg       0.95       0.94       0.94         114
weighted avg       0.95       0.95       0.95         114
```

Gambar 2.14. PCA

### 2.15 Perbandingan Model dengan dan tanpa PCA

Jumlah fitur berkurang dari 30 menjadi 2, Akurasi tetap stabil, Model dengan PCA lebih sederhana dan efisien

```
hasil = pd.DataFrame({
    'Model': ['SVM tanpa PCA', 'SVM dengan PCA (2 Komponen)'],
    'Jumlah Fitur': [X.shape[1], X_train_pca.shape[1]],
    'Akurasi': [acc_no_pca, acc_pca]
})
```

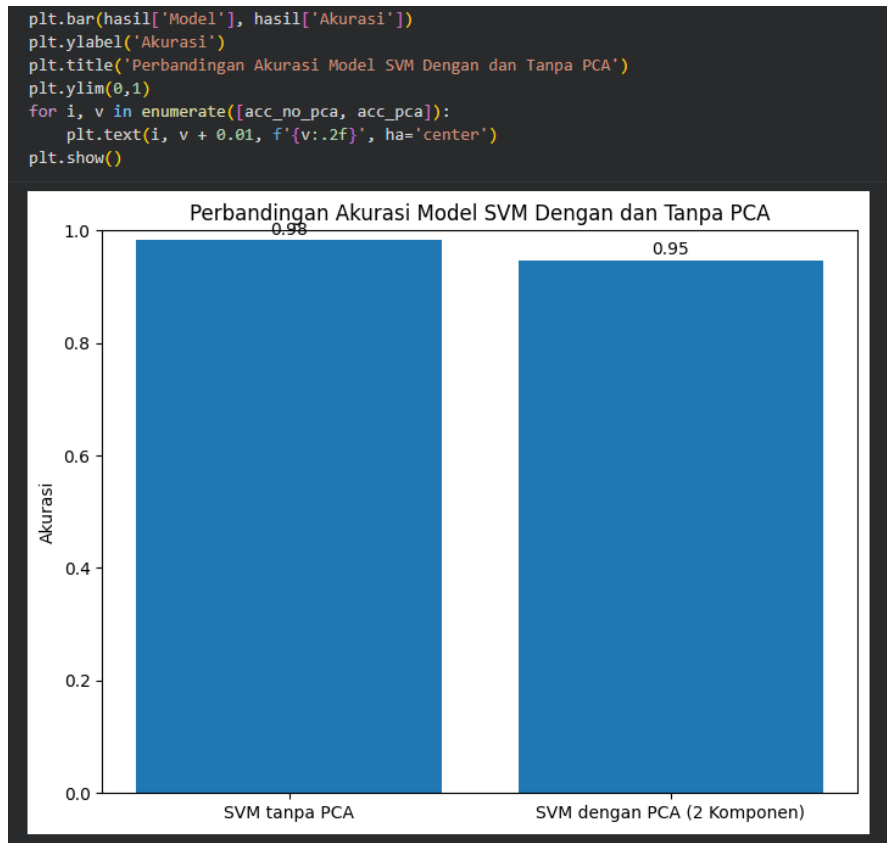
```
hasil
```

|   | Model                       | Jumlah Fitur | Akurasi  |
|---|-----------------------------|--------------|----------|
| 0 | SVM tanpa PCA               | 30           | 0.982456 |
| 1 | SVM dengan PCA (2 Komponen) | 2            | 0.947368 |

Gambar 2.15. Perbandingan SVM dengan dan tanpa PCA

### 2.16 Visualisasi Perbandingan

Penerapan PCA pada dataset Breast Cancer berhasil menyederhanakan data berdimensi tinggi tanpa mengorbankan performa klasifikasi. PCA terbukti efektif untuk reduksi dimensi, visualisasi data, dan peningkatan efisiensi model SVM.



**Gambar 2.16.** Visualisasi Perbandingan

## KESIMPULAN

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa dataset Breast Cancer merupakan dataset medis berdimensi tinggi dengan jumlah fitur yang cukup banyak, sehingga berpotensi meningkatkan kompleksitas model klasifikasi. Kondisi ini menunjukkan pentingnya penerapan teknik reduksi dimensi agar data dapat dianalisis secara lebih efisien tanpa kehilangan informasi yang relevan.

Penerapan Principal Component Analysis (PCA) pada dataset ini terbukti mampu mereduksi jumlah fitur secara signifikan dari 30 fitur menjadi dua komponen utama. Meskipun terjadi penyederhanaan data, informasi penting tetap dapat dipertahankan, yang ditunjukkan melalui visualisasi PCA dua dimensi serta hasil klasifikasi yang masih mampu membedakan kelas *benign* dan *malignant* dengan cukup baik.

Hasil perbandingan model menunjukkan bahwa performa Support Vector Machine (SVM) dengan PCA relatif sebanding dengan model SVM tanpa PCA. Hal ini menandakan bahwa penggunaan PCA tidak menurunkan akurasi model secara signifikan, namun justru memberikan keuntungan dari sisi efisiensi dan kemudahan analisis. Dengan demikian, PCA dapat disimpulkan sebagai metode yang efektif untuk menangani dataset medis berdimensi tinggi, baik untuk tujuan visualisasi maupun optimasi model klasifikasi.



## Referensi:

scikit-learn Developers

Breast Cancer Dataset — sklearn.datasets

[https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load\\_breast\\_cancer.html](https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_breast_cancer.html)

Kamyar Azar

Breast Cancer Classification Using PCA in Python

<https://www.kaggle.com/code/kamyarazar/breast-cancer-classification-using-pca-in-python>

UCI Machine Learning Repository

Breast Cancer Wisconsin (Diagnostic) Dataset

[https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+\(Diagnostic\)](https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+(Diagnostic))